

Sistema Game-Con Bahia Pocket

Marcus Vinícius Leite dos Santos

UEFS– Universidade Estadual de Feira de Santana
Av. Transnordestina, s/n, Novo Horizonte
Feira de Santana– BA, Brasil– 44036-900

marcusleiteacc@gmail.com

Abstract. *The organization of Game-Con Bahia Pocket developed a ticket sales system in Python to assist the event's box office at CUCA. The project consists of a management algorithm that controls venue capacity, classifies attendees into different tariff categories, and processes physical cash payments, including change calculation and handling insufficient funds. The program is based on conditional structures and repetition loops to record each sale, accumulate financial data, and validate user inputs. Finally, the system generates detailed statistics on total revenue and public attendance, proving to be an effective tool for the logistical and administrative control of the convention.*

Resumo. *A organização da Game-Con Bahia Pocket desenvolveu um sistema de venda de ingressos em Python para auxiliar a portaria do evento no CUCA. O projeto consiste em um algoritmo de gestão que controla a lotação do espaço, classifica os participantes em diferentes categorias tarifárias e processa pagamentos em dinheiro físico, incluindo o cálculo de troco e o tratamento de saldos insuficientes. O programa é fundamentado em estruturas condicionais e laços de repetição para registrar cada venda, acumular dados financeiros e validar as entradas do usuário. Por fim, o sistema gera estatísticas detalhadas sobre a arrecadação e o público, demonstrando ser uma ferramenta eficaz para o controle logístico e administrativo da convenção.*

1. Introdução

A Game-Con Bahia com o passar dos anos veio se tornando um evento anual marcante para o público da cidade de Feira de Santana. No ano de 2026, a última edição destacou-se por superar as expectativas de participantes do concurso de cosplay, se tornando a atração mais concorrida do evento. Devido a movimentação e demanda por mais acontecimentos parecidos, a organização decidiu criar uma versão reduzida do evento, nomeada como “Game-Con Bahia Pocket”, feita com o objetivo exclusivo de atender ao concurso de cosplay e manter o público engajado ao longo do ano sem ter que lidar com os mesmos custos da edição principal.

A Pocket vai acontecer no CUCA, o Centro Universitário de Cultura e Arte administrado pela UEFS. Como parte do acordo entre a organização e universidade, ficou combinado que os estudantes teriam um valor de entrada reduzido, como forma de incentivar a participação. Nas reuniões de planejamento, a coordenação do evento levantou uma série de pontos a serem resolvidos antes da estreia.

- Crianças até 9 anos não pagam a entrada

- O público de 10 a 17 anos paga R\$15,00 de entrada, como um valor simbólico ao público jovem.
- Adultos, a partir de 18 anos, pagam R\$40,00 de entrada.
- Universitários da UEFS, maiores de idade, pagam meia-entrada, R\$20,00, desde que apresentem comprovante de matrícula contendo precisamente 9 dígitos numéricos. Caso a matrícula não seja conferida, a pessoa pagará o valor da entrada de adulto normal.

A Pocket não vai dispor de máquinas ou totem de pagamento eletrônico, contando somente com o pagamento em dinheiro físico feito através de um vendedor da organização na portaria do CUCA e, portanto, a disposição de um totem destinado somente a auxiliar o vendedor é de grande importância para o controle da logística e agilidade da venda de ingressos para a convenção.

Diante desse cenário, sintetiza-se que a venda dos ingressos do evento requer tratar de atender os clientes de forma controlada e segura enquanto não seja lenta a ponto de atrapalhar a dinâmica da convenção. Portanto, é necessário um sistema que abarque todas as funções de gestão de dados para servir de apoio ao funcionário da portaria.

Com base nesse pretexto, o programa desenvolvido em Python tem como o objetivo sanar os eventuais problemas que podem acontecer com o controle de lotação, pagamento e classificação feita pelo vendedor, sendo capaz de cobrar cada pessoa conforme a categoria informada, calcular o troco correto para cada pagamento, avisar o vendedor caso o dinheiro não seja o suficiente para o pagamento e quando a lotação do espaço tiver se esgotado e permitir o encerramento do sistema a qualquer momento, através da entrada de um código de encerramento previamente combinado. Além disso, após o encerramento do dia, o programa auxiliará o setor estatístico da organização informando a arrecadação total da edição, suas especificidades e o prêmio total e repartido aos vencedores do concurso.

2. Fundamentação Teórica

Para a construção do programa Python, foram utilizadas estruturas de lógica de programação destinadas a atender cada requisito do problema:

• Estruturas condicionais:

-If Else / Match Case: Estruturas baseadas no mesmo conceito, o If(Else e Elif) e Match Case são destinadas a realizar determinada parte de um algoritmo SE uma ou mais determinadas condições forem atendidas [Python Software Foundation 2024]. Usada no processo de verificações e opções de escolha.

• Estruturas de repetição:

-While: Estrutura voltada para repetir determinada parte de um algoritmo ENQUANTO determinada condição continue sendo verdadeira [Python Software

Foundation 2024]. Usada para manter o looping principal do Menu principal e espera de entradas do usuário até receber um dado válido.

-For: Estrutura com o mesmo propósito da anterior, mas limitada PARA um número determinado de repetições [Python Software Foundation 2024]. Podendo se tornar flexível para o percorrendo de strings, foi utilizada para analisar a entrada do pagamento do usuário e conferir se há alguma inconsistência nos caracteres digitados.

- **Funções Built-in do Python:**

Primeiramente, funções built-in são comandos/ferramentas embutidas junto a linguagem Python para realizar determinada ação preestabelecida. Exs: print(), input(), int(), range().

-IsDigit: Função de verificação de uma string que retorna True caso todos os caracteres sejam dígitos [Python Software Foundation 2024]. Utilizada para verificar a entrada de quantidades, identificação de matrícula e opções digitadas pelo usuário.

-Lower: Função de conversão de caracteres maiúsculos para minúsculos dentro de string, criando-se uma nova string que pode ser usada para evitar as possibilidades que o usuário pode variar a "caixa alta" de uma sequência de caracteres [Python Software Foundation 2024]. Utilizada na sessão de pagamento, onde o operador deve digitar se o cliente deseja COMPLETAR o saldo necessário para pagar pelo ingresso ou CANCELAR a compra.

3. Metodologia

Esse projeto foi desenvolvido com base na metodologia PBL (Problem-Base Learning), buscando solucionar o problema por meio de sessões de discussões em grupo. O tópico discutido em nossa primeira reunião foi a estruturação do problema para uma interface destinada ao vendedor operando, levantando aspectos necessários e desnecessários para o funcionamento do sistema, como discutir se o programa rodaria somente um dia de convenção por vez que fosse iniciado ou contabilizaria os dias a cada encerramento. Dessa maneira, modelamos a ideia principal para ser possível já trazer um escopo inicial para as próximas sessões.

Para guiar a ordem de codificação e garantir a aplicação das estruturas de escolha e os loopings de repetição, o algoritmo foi desenhado de forma gráfica através de um fluxo completo de execução do sistema pode ser visualizado no **Anexo 1. Fluxograma.**

Sobre o processo de desenvolvimento do software, a seguir os pontos importantes para sua construção:

- **Definição de Requisitos e Funcionalidades:**

1. Variáveis de controle de fluxo: O programa deve apresentar variáveis *publico* e *encerramento* inicializadas para manter os loopings enquanto a locação ou o código de encerramento não forem definidos.

```

publico = False

while publico == False:
    publico_maximo = input("Defina o público máximo do evento: ")

    if publico_maximo.isdigit():
        publico_maximo = int(publico_maximo)
        publico = True
    else:
        print("Entrada incorreta.")

```

Figura 1. Inicialização de variavel publico e definição de público máximo.

```

encerramento = False

#MANTÉM O PROGRAMA RODANDO ENQUANTO O OPERADOR
#NÃO ENCERRAR O PROGRAMA COM O COD. ENCERRAMENTO
> while not encerramento: ...

```

Figura 2. Looping enquanto encerramento for Falso.

2. Opções de escolha do usuário: Deve apresentar opções de cadastro ou encerramento do programa, seleção da categoria e opção de completar ou cancelar um pagamento insuficiente. Todas explicitamente apresentadas para o usuário e facilitar o uso do programa;
3. Condição de saldo insuficiente: Permitir para que caso o cliente não tenha apresentado saldo suficiente para o ingresso, possa completar o valor ou cancelar a compra;
4. Composição das estatísticas: Estruturar as condições e comparações dos dados obtidos durante o programa às suas estatísticas;
5. Monitoração dos erros de entrada: Verificar do tipo do dado e validação de sua composição de acordo ao processamento a ser solicitado. Ex.: Verificar se a entrada recebimento, valor de pagamento recebido pelo cliente, é composto por números ou vírgula/ponto decimal e se ela começa com número e possui somente uma vírgula/ponto decimal no máximo.

- **Ordem de Codificação**

Inicialmente, o plano foi transpor o problema para uma versão funcional do algoritmo. O primeiro passo foi inicializar as variáveis mais claras do problema (1. Quantidade de adultos, jovens, crianças e universitários; 2. Orçamento incrementado a cada nova inscrição; 3. Código de encerramento.), definir um limite de locação assim que o programa for iniciado e codificar um menu com interação do operador para inscrever novos participantes enquanto o público máximo não for atingido, ilustrado na Figura 3 e Figura 4.

Dado a base do algoritmo de contabilizar a quantidade de cada categoria inscrita e o orçamento arrecadado como feito, também foi adicionado o processamento desses dados para a exibição das estatísticas de: Público total; Vagas restantes; Ticket médio incluindo e não incluindo crianças; Arrecadação total e de cada categoria; Categoria pagante que mais e menos arrecadou para o evento; Premiação total e para cada colocação.

Em seguida, o próximo requisito tratado foi flexibilização do programa com as diferentes quantidades de pagamento que possa receber, considerando a entrada de um valor maior, menor ou igual ao cobrado para a categoria do cliente. Como especificado pelo problema, o cliente tem a escolha de completar o saldo insuficiente ou cancelar a compra, logo foi adicionado nessa etapa um controle condicional destinado a manter o cliente no looping de pagamento ou não pagar, retornar ao menu e retirar o valor já adicionado caso ele escolha completar ou não, ilustrado na Figura 5.

Por fim, o último passo foi a verificação das entradas do usuário, certificando que o programa vai rodar funcionalmente até em casos de acidentes ou erros do operador, garantindo a segurança do sistema, ilustrado na Figura 6. Para isso, foi utilizada a aplicação de estruturas condicionais junto a funções booleanas para conferir o tipo de dado das entradas da locação, opção do menu principal, categoria e valor de pagamento e outras especificações de entrada. Afinal, essa ordenação foi favorável ao projeto pois permitiu escupir o sistema, analisando e podendo implementar soluções para a maior parte dos casos de atendimento do vendedor e as fragilidades de erros que tanto ele quanto o cliente possam cometer sobre a lógica do algoritmo.

O sistema operacional utilizado na construção desse programa foi o Windows 10, utilizando a Linguagem Python, versão 3.14, através do Ambiente de Desenvolvimento Integrado(IDE) Visual Studio Code, instalado por meio de sua página. Além disso, foram utilizadas as extensões do VSCode: 1.Code Runner, versão 0.12.2; 2.Pylance, versão 2026.3.1; 3.Python, versão 2026.4.0. Todas as extensões instaladas na própria IDA como ferramentas voltadas para o desenvolvimento da solução.

```

publico = False

while publico == False:
    publico_maximo = input("Defina o público máximo do evento: ")

    if publico_maximo.isdigit():
        publico_maximo = int(publico_maximo)
        publico = True
    else:
        print("Entrada incorreta.")

pessoas_inscritas = 0
adultos = jovens = universitarios = crianacas = 0
orcamento = 0
cod_encerramento = "5020jzp"
print(f"Cod.Encerramento -> {cod_encerramento}\n\n")

```

Figura 3. Inicialização de variáveis fundamentais e limite de locação.

```

while not encerramento:
    print(f"\n|Menu de Opções|\n
          |  \n1-Inscrição\
          |  \n2-Encerrar Programa\
          |  \n\nVagas restantes: {publico_maximo-pessoas_inscritas}\n")
    opcao = input("Insira qual das opções deseja realizar: ")

    if opcao.isdigit():
        opcao = int(opcao)
        match opcao:
>         case 1: ...
>         case 2: |...
            case _:
                print("Opção inválida.")
    else:
        print("Entrada incorreta.")

```

Figura 4. Menu do operador.

```

recebimento = float(recebimento)
if recebimento > cobrado:
    print(f"Troco: R${recebimento - cobrado:.2f}")
    pagamento = True
elif recebimento < cobrado:
    print(f"Pagamento insuficiente. R${cobrado-recebimento:.2f} para completar")
    opcao_completar = ""
    while opcao_completar.lower() not in("completar","cancelar"):
        opcao_completar = input("Deseja completar o valor ou cancelar o pagamento: ")
        if opcao_completar.lower() == "completar":
            orcamento += recebimento
            cobrado -= recebimento
            valor_completado += recebimento
        elif opcao_completar.lower() == "cancelar":
            orcamento -= valor_completado
            encerrar_pagamento = True
        else:
            print("Entrada incorreta.")
    else:
        pagamento = True

```

Figura 5. Estrutura do pagamento insuficiente.

<pre> if publico_maximo.isdigit(): publico_maximo = int(publico_maximo) publico = True else: print("Entrada incorreta.") </pre>	<pre> elif recebimento < cobrado: print(f"Pagamento insuficiente. R\${cobrado-recebimento:.2f} para completar") opcao_completar = "" while opcao_completar.lower() not in("completar","cancelar"): opcao_completar = input("Deseja completar o valor ou cancelar o pagamento: ") if opcao_completar.lower() == "completar": orcamento += recebimento cobrado -= recebimento valor_completado += recebimento elif opcao_completar.lower() == "cancelar": orcamento -= valor_completado encerrar_pagamento = True else: print("Entrada incorreta.") else: pagamento = True </pre>
<pre> if matricula.isdigit() and len(matricula) == 10: categoria = "universitario" universitario = True print("Categoria: Universitário") cobrado = 20 </pre>	

Figura 6. Validação de entradas.

4. Resultados e Discussões

- **Manual de Uso**

Para utilizar o programa de inscrição da Game-Con Bahia Pocket, o sistema roda diretamente pelo terminal, implementado com instruções didáticas e menus explicativos para cada uma de suas funcionalidades. Através desses menus, o operador escolhe se deseja inscrever um novo participante ou encerrar o programa e qual categoria deseja enquadrar o participante atendido.

As principais ações para o operador incluem:

1. Definição de público máximo do evento. Solicitado no início do programa

```
Defina o público máximo do evento: 10
```

Figura 7. Definição de público máximo do evento: 10.

2. Escolha no menu de opções principal:

```
|Menu de Opções|
1-Inscrição
2-Encerrar Programa

Vagas restantes: 10

Insira qual das opções deseja realizar: 1
```

Figura 8. Menu de opções, 10 vagas restantes.

3. Cadastro no menu de categoria:

```
|CATEGORIAS|
1-Adultos - R$40,00
2-Jovens de 10 a 17 anos - R$15,00
3-Universitários - R$20,00
4-Crianças até 9 anos - Isentos de pagamento

|INSCRIÇÃO|

Insira qual das categorias o participante se enquadra na inscrição: █
```

Figura 9. Categorias, valores do ingresso e seleção de categoria.

4. Inserir valor do pagamento e optar entre completar ou cancelar a transação em caso de saldo insuficiente.

```
Categoria: Adulto
Valor recebido pelo participante: 37
Pagamento insuficiente. R$3.00 para completar
Deseja completar o valor ou cancelar o pagamento: completar
Valor recebido pelo participante: █
```

Figura 10. Caso de saldo insuficiente.

5. Inserir código de encerramento.

```
|Menu de Opções|
1-Inscrição
2-Encerrar Programa

Vagas restantes: 9

Insira qual das opções deseja realizar: 2
Insira o código de encerramento do sistema: █
```

Figura 11. Código de encerramento.

- **Testes Realizados**

Foram realizados testes para conferir o correto processamento das entradas sem algum erro de processamento de dados, como:

- Inserção de um caractere ao invés de número para o público máximo, opção, matrícula e valor. Para os casos do valor conferiu-se a quantidade de vírgulas e pontos e para a matrícula se possui exatamente 9 dígitos numéricos.

```
Insira qual das categorias o participante se enquadra na inscrição: 1
Categoria: Adulto
Valor recebido pelo participante: .
Entrada incorreta.
```

```
Insira qual das opções deseja realizar: a
Entrada incorreta.
```

```
Insira o código de encerramento do sistema: .
Código inválido.
```

```
Defina o público máximo do evento: a
Entrada incorreta.
```

```
Deseja completar o valor ou cancelar o pagamento: a
Entrada incorreta.
```

Figura 12. Entradas incorretas e inválidas.

- Inclusão de casos de empate de arrecadamentos ou situação de público zero na estatística das categorias que mais e menos contribuíram.

```
Categoria pagante que mais arrecadou: Adultos;
Categoria pagante que menos arrecadou: Jovens e Universitários - Empatados;
```

```
Categoria pagante que mais arrecadou: Público Zero - Nenhuma das Categorias;
```

Figura 13. Caso de empate de arrecadamento e público zero.

Por ser um problema mais simples, a construção do programa não apresentou muitas ou grandes dificuldades. Durante as sessões PBL, a dúvida mais duradoura foi entender de qual maneira seria capaz verificar a entrada de números, valores monetários e identificação de uma matrícula válida e entender como os artifícios e alternativas para solucionar essa dúvida funcionariam, mas mesmo assim sem grandes desafios. No geral o desenvolvimento desse caixa foi uma boa prática tanto técnica quanto metodológica, permitindo que eu aplicasse e aprimorasse meus conhecimentos em programação e interpretação de problemas e aprendesse sobre a construção de fluxogramas e escrita de relatórios.

5. Conclusão

Com isso, avalio que o programa desenvolvido alcançou os requisitos mínimos e atendeu ao problema apresentado, garantindo uma gestão de dados e regras de negócio que auxiliem o trabalho do vendedor da Game-Con Bahia Pocket. Imagino que seu uso seja o suficiente para fornecer uma assistência técnica a administração do evento com a precisão de suas estatísticas e processamentos. Como ponto fraco, vale ressaltar que o sistema não possui uma forma de retornar um pagamento depois de confirmado, não cobrando a possibilidade do operador inserir um valor igual ou superior ao cobrado quando o cliente não tiver o suficiente. Posteriormente o programa pode ser incrementado com presença de outras formas de pagamento via Pix, Cartão de Crédito ou Débito através de APIs, para evitar erros de valores que eventualmente o vendedor pode acabar cometendo.

6. Referências Bibliográficas

Python Software Foundation. if, for, while & match statements. Online; acessado 04 de setembro de 2026. URL: https://docs.python.org/pt-br/3/reference/compound_stmts.html.

Python Software Foundation. str.isdigit(). Online; acessado 04 de setembro de 2026. URL: <https://docs.python.org/pt-br/3/library/stdtypes.html#str.isdigit>.

Python Software Foundation. str.lower(). Online; acessado 04 de setembro de 2026. URL: <https://docs.python.org/pt-br/3/library/stdtypes.html#str.lower>.

Microsoft. Visual Studio Code [software]. 2024, Disponível em: <https://code.visualstudio.com/>. Acesso em: 05 SET. 2026.

Anexo 1. Fluxograma

